



Research Software that others can find, trust and run.

From shared science in Vensim and
Stella to inspectable, citable research
artifacts — in 90 minutes.

AUTHORS

Raphael Ginster · Matthias Papesch
Vasiliy Seibert
Lecturer: Vasiliy Seibert · [ORCID](#)

RUNNING EXAMPLE

[FAIR-systemDynamics/fair-hunter-prey-simulation](#)

Concept DOI: awaiting first release

SESSION

90 min · 57 core slides · EN
6 optional walkthroughs
Lecture: CC BY 4.0 · model evidence: CC BY-NC-SA 4.0

Four lectures, one story

This fourth lecture pivots from *research data* to *research software*. Software runs, depends, versions, breaks — it needs its own playbook.

LECTURE 1 · MOTIVATION

FAIR principles, the data lifecycle, why we care about DOIs and ORCIDs.

LECTURE 2 · PLANNING IN RDMO

Writing a Software/Data Management Plan in `rdmo.nfdi4ing.de`.

LECTURE 3 · METADATA

Describing data with Metadata4Ing, RO-Crate, controlled vocabularies.

Today (Lecture 4): the same mindset, for the *code* that produced the data. The community answer is **FAIR4RS** — *FAIR Principles for Research Software*.

Agenda · 3 × theory ↔ 3 × practical

Theory blocks introduce a FAIR4RS letter; each is immediately followed by a hands-on practical on an NFDI4ING service.

TIME	BLOCK	FORMAT	NFDI4ING SERVICE	FAIR4RS
0:00–0:10	Opening, recap & shared model	Theory	—	—
0:10–0:22	Findable & Accessible	Theory	—	F1 F2 F3 F4 A1 A2
0:22–0:34	Practical 1 · Discover the hunter–prey model	Hands-on	Betty Research Engine	F A
0:34–0:46	Interoperable	Theory	—	I1 I2
0:46–1:00	Practical 2 · Test interoperability	Hands-on	Jupyter Service	I
1:00–1:12	Reusable	Theory	(back-ref RDMO SMP)	R1 R2 R3
1:12–1:26	Practical 3 · Capture reusability via RO-Crate	Hands-on	Jupyter Service	R
1:26–1:30	Wrap-up, FAIR4RS checklist	Theory	—	—

Why make research software FAIR?

FAIR Principles for Research Software
(FAIR4RS Principles)

RDA Recommendation



Software runs — and once it runs, *people depend on it*. Three concrete things you gain when your code is FAIR:

① CITATIONS & CREDIT

A DOI makes your software *citable like a paper*. Citing the archived version gives authors credit. Citation tracking depends on authors, records, and indexing services.

② REPRODUCIBILITY ON A TIMESCALE OF YEARS

Pinned dependencies + an archived snapshot preserve the context needed to attempt the same computation years later. Reviewers, funders, and future-you all benefit.

③ COLLABORATION WITHOUT EMAIL

A stranger can `pip install` your code, run an example, and open a pull request — without asking you for a zip file. Contribution ≈ free.

But software isn't "just another dataset"

DATA · STATIC	SOFTWARE · DYNAMIC
Bytes at rest	Executes — needs an environment
Linear snapshots	Depends on other moving software
Immutability is a feature	Versions at many granularities
Described by metadata, done	Rots — breaks without being touched

Hence **FAIR4RS** — the RDA-endorsed re-interpretation of FAIR where every letter is rewritten for things that *run*.

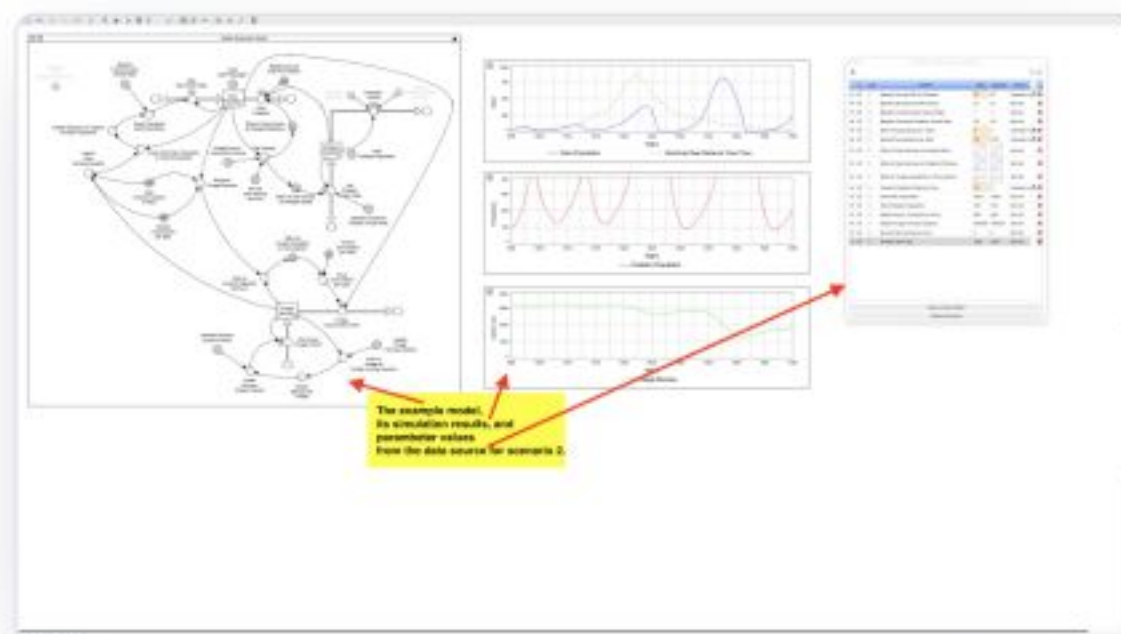
Chue Hong et al., *FAIR Principles for Research Software*, RDA, 2022. doi.org/10.15497/RDA00068

One repo, running through every slide

fair-hunter-prey shared model

A Kaibab ecosystem model implemented in Vensim and Stella Architect. Both implementations share a scientific specification and two cases; their file formats and numerical execution differ.

Every theory slide connects a FAIR4RS concept to evidence in this repository — from the CI badge to the Zenodo release workflow and CITATION.cff.



Existing native evidence; no new native execution. [Inspect full-size screenshot](#)

- GitHub · fair-hunter-prey-simulation
- Concept DOI: awaiting first release

Review-branch repository evidence. DOI publication follows approval of the exact release commit. [Inspect full-size screenshot](#)

[Runnable examples](#) · [Semantic model and vocabulary](#)

Shared scientific specification

The literature defines an ecosystem of **deer, predators and forage**. Equations, lookup functions, units and initial conditions describe the scientific problem.



The existing semantic model connects the specification to both native implementations. [Open full-size image](#)

[Open the unchanged semantic model](#) · [Original scientific specification](#)

[Science](#) →

Vensim / Stella →

Case 1 / Case 2 →

Execution →

Inspection →

FAIR record

Shared use cases, tool-specific implementations

USE CASE	VENSIM IMPLEMENTATION	STELLA ARCHITECT IMPLEMENTATION
Define	Equations and sketch in .mdl	Equations and views in .stm (XML/XMILE)
View	Native view or exported HTML/SVG	Native view or exported PDF/SVG
Document / comment	Model text, variable descriptions, report	Model documentation, equations, screenshots
Edit	Vensim authoring tools	Stella Architect authoring tools
Configure	Existing .cin, CSV and workbook inputs	Parameter, lookup and historical-data CSVs
Execute	Native Vensim engine; separate PySD route	Native Stella engine; separate PySD route
Export	Source, documentation and result data	Source, documentation and result data

Tool-specific artifacts and access

ARTIFACT	WHAT IT CARRIES	ACCESS BOUNDARY
.mdl / .stmx	Editable model declarations; .stmx is XML/XMILE	Readable source, tool-specific semantics
CSV / HTML / SVG / PDF	Exported values or documentation	Open inspection in appropriate readers
.vpmx	Vensim packaged model	Vendor runtime; an export option, absent here
ZIP / .stmz	Stella bundled model and supporting files	Container only; an export option, absent here

Proprietary does not always mean paid: [Vensim Model Reader](#) and [isee Player](#) offer restricted viewing/runtime workflows. Compatibility with this snapshot was not newly tested.

Case 1 and Case 2

PARAMETER	CASE 1	CASE 2
Annual kills per predator	40	20
Carrying-capacity horizon	2 years	4 years
Desired consumption per deer	0.75 t/(deer·year)	0.5 t/(deer·year)
Predator removal fraction	0/year	0.2/year

The outcome is not an isolated predator-removal effect.

Vensim: [Case 1 parameters](#) + initial-stock workbook; [Case 2 changes](#).

Stella: [Case 1 parameters](#) / [Case 2 parameters](#).

Open inspection and independent execution

Inspect saved exports

- Read existing Vensim and Stella CSVs.
- Use pandas and Matplotlib for tables and figures.
- No simulation or proprietary application is executed.

[PySD supports translation and simulation, with feature limitations](#). Stella Case 2's native CSV has final values only; a new PySD trajectory is separate evidence.

Run an implementation with PySD

- Translate a copy of .mdl or .stm.
- Apply the existing numerical adapters.
- Compare the new run with its own tool's reference.

01

Findable & Accessible research software

Making software **publishable** and **retrievable** by both humans and machines.

F**A**

I

R

F1 · Globally unique and persistent identifier

F1

F1.1

F1.2

FAIR4RS · F1

Software is assigned a globally unique and persistent identifier.

PLAIN ENGLISH · Every piece of software needs an ID that (a) no other object in the world shares, and (b) is maintained for persistent resolution — so humans and machines can point at it without ambiguity.

- **F1.1** — components at different granularities (project, release, function) each get distinct IDs.
- **F1.2** — each *version* gets its own distinct ID.

IN FAIR-HUNTER-PREY

- ✓ **Concept DOI:** [Concept DOI: awaiting first release](#) → F1 / F1.1.
- ✓ **Version DOI:** [Version DOI: awaiting first release](#) → F1.2.
- ✓ **v0.7.0** identifies the first planned archived software release; prior teaching tags remain distinct.
- ✓ Identify the scientific model, Vensim/Stella source, Case 1/2 configuration, and each recorded run separately.

One project · distinct versions · traceable research objects.

F1 · A DOI is a *promise*, not a URL F1

Anatomy

doi.org / 10.5281 / zenodo.19677548

- 10.5281 — registrant prefix (Zenodo / DataCite)
- zenodo.19677548 — suffix, opaque to DOI resolver
- doi.org/... — HTTP resolver, not the identity

A DOI is designed for long-term resolution through maintained registration and repository services, independently of a GitHub URL.

Anatomy example: the published awesome-sim DOI. The hunter-prey DOI is issued after the reviewed release.

How you get one · live demonstration

- ✓ Connect the repository in [Zenodo's GitHub settings](#). The switch installs the release webhook.
- ✓ Review authors, licenses and `.zenodo.json`; approve the exact source commit for `v0.7.0`.
- ✓ Create a GitHub release from that commit. Verify the delivery and Zenodo record before claiming a DOI.
- ✓ Keep main and the live semantic site unchanged; the release uses the reviewed branch snapshot.

• Zenodo · GitHub settings

git → GitHub → Zenodo

text

```
reviewed commit → GitHub release  
→ release webhook → Zenodo archive  
→ verify concept DOI + version DOI
```

[Release checklist](#) · [Concept DOI: awaiting first release](#)

One project, many versions — each citable F1.1 F1.2

Two levels of DOI

- **Concept DOI** — the project across all versions; always resolves to the latest release.
- **Version DOI** — a specific snapshot; identifies one archived version.

Cite this version, not the project when reporting a computation — so readers can identify the exact archived version and its inputs.

Git teaching milestones `v0.1-scaffold` through `v0.6-verification` document development. They are not Zenodo deposits.

Hunter–prey release identities

LEVEL	IDENTIFIER	MEANING
Concept	Concept DOI: awaiting first release	Project across archived versions
v0.7.0	Version DOI: awaiting first release	Exact reviewed software snapshot
Execution	Implementation + case + source hash	A particular PySD run

A single release establishes both a project-level and a version-level identifier. Future releases receive distinct version DOIs.

F2 · F3 · F4 · rich, linked, harvestable metadata

FAIR4RS · F2 · F3 · F4

F2. Software is described with rich metadata.

F3. Metadata clearly and explicitly include the identifier of the software they describe.

F4. Metadata are FAIR, searchable and indexable.

PLAIN ENGLISH · The metadata itself must be findable (not buried in a PDF), it must link back to the software's DOI, and it must be consumable by the registries and search engines that harvest it.

IN FAIR-HUNTER-PREY

- ✓ [CITATION.cff](#) — GitHub's citation entry point.
- ✓ [codemeta.json](#) — structured software metadata and qualified references.
- ✓ [.zenodo.json](#) — creators, version, licensing and related literature for Zenodo.
- ✓ DOI and CI badges make publication and verification evidence visible. Publication status: **awaiting-commit-review**.

Metadata supports discovery; actual indexing and search results must be checked.

Machines can only find what they can *read* F2 F3 F4

CITATION.cff

YAML · CFF 1.2.0

```
cff-version: 1.2.0
type: software
title: fair-hunter-prey-simulation
version: 0.7.0
authors:
  - family-names: Ginster
    given-names: Raphael
  - family-names: Papesch
    given-names: Matthias
  - family-names: Seibert
    given-names: Vasiliy
# Full file: identifiers, licenses, references
```

[Read the complete metadata](#) · Zenodo uses .zenodo.json when both files are present.

codemeta.json

JSON-LD · CodeMeta 2.0

```
{
  "@context": "https://doi.org/10.5063/schema/codemeta-2.0",
  "@type": "SoftwareSourceCode",
  "name": "fair-hunter-prey-simulation",
  "version": "0.7.0",
  "programmingLanguage": "Python",
  "runtimePlatform": [
    "Python 3.12",
    "Python 3.13"
  ],
  "softwareRequirements": [
    "pysd==3.14.3",
    "pandas>=2.2,<3"
  ]
}
```

[Read the complete metadata](#) · Zenodo uses .zenodo.json when both files are present.

A1 · A1.1 · retrievable via a standardized protocol A1 A1.1

FAIR4RS · A1 · A1.1

A1. Software is retrievable by its identifier using a standardized communications protocol.

A1.1. The protocol is open, free, and universally implementable.

PLAIN ENGLISH · Given the DOI or the git URL, any human or machine must be able to fetch the code over a protocol that is publicly documented, costs nothing to implement, and has at least one open-source client.

IN FAIR-HUNTER-PREY

- ✓ `git clone` over HTTPS retrieves models, configurations, readers and history.
- ✓ `pip install` installs the package and PySD execution route.
- ✓ Zenodo archive retrieval becomes available after the first approved release.
- ✓ Text, CSV, HTML and SVG can be inspected openly. Native editing and execution depend on the relevant vendor tool.

Standard retrieval protocols and native application access are separate questions.

Open, free, universally implementable A1 A1.1

FAIR4RS **A1.1** asks: *"Is the protocol open, free, and universally implementable?"* For research software, the answer is almost always **HTTPS + git**.

retrieve fair-hunter-prey, three ways

```
# (a) clone - full history
git clone https://github.com/FAIR-systemDynamics/fair-hunter-prey-simulation

# (b) install the exact reviewed snapshot
python -m pip install "fair-hunter-prey[teaching] @ git+https://github.com/FAIR-systemDynamics/fair-hunter-prey-simulation@ca6c7c85df3335e7197b1ef1bfd75af2ea41a33a"

# (c) archived release
# Zenodo archive: pending reviewed release
```

The protocol matters

- **HTTPS** — an IETF standard, any device, no account.
- **git** — an open protocol with multiple interop implementations.
- **PyPI / conda** — open registries using the same stack.

Private FTP? Proprietary portal requiring a PDF application?
Not A1.1.

← LECTURE 1 · DOI + ORCID

A1.2 · auth is a *feature*, not a gate A1.2

FAIR4RS · A1.2

The protocol allows for an authentication and authorization procedure, where necessary.

PLAIN ENGLISH · Authentication is a *capability* of the protocol, to be used only when the content genuinely requires it (private review, rate-limits, contribution provenance). A public repo that *optionally* supports OAuth is still fully A-compliant.

IN FAIR-HUNTER-PREY

- ✓ Public model sources and saved results can be retrieved without login.
- ✓ GitHub authentication supports contributions and release administration.
- ✓ Zenodo connects through GitHub OAuth; the hosted Jupyter service has its own sign-in.
- ✓ Vendor applications have separate access terms. Proprietary software does not always mean paid access.

Authentication, where genuinely necessary A1.2

What A1.2 is *not*

- It is **not** an excuse to hide public research behind a login.
- It is **not** about protecting code from reviewers.

What A1.2 *is*

- A **Personal Access Token** for a private fork during peer review.
- **OAuth** (GitHub login to Zenodo, to the Betty Engine, ...) — authenticated *access*, not locked content.
- Rate-limit mitigation & contribution provenance (who pushed what).

cloning with a PAT (private review)

```
gh auth login
gh repo clone OWNER/PRIVATE-REPOSITORY

# CI: use the workflow-scoped GITHUB_TOKEN
# for GitHub operations; OIDC for supported
# external cloud identity federation.
```

Practical 1 uses **OAuth** to log into the Betty Research Engine via GitHub — you will see A1.2 in action.

A2 · metadata outlives the software A2

FAIR4RS · A2

Metadata are accessible, even when the software is no longer available.

PLAIN ENGLISH · Software rots. Hosts go bankrupt. Dependencies disappear. But the *record that the software existed* — who wrote it, what it did, what it depended on — must stay resolvable, so citations and reproducibility claims don't silently break.

IN FAIR-HUNTER-PREY

- ✓ Archive a reviewed source snapshot with authors, licenses and environment information.
- ✓ Preserve citation metadata even if future native software access changes.
- ✓ Check Zenodo deposit and Software Heritage ingestion separately; record only observed identifiers.
- ✓ Keep the original saved results inspectable alongside the model sources.

[Publication and preservation checklist](#)

Metadata outlives the software A2

A2: if the software disappears, the metadata should remain accessible. Two instruments do the heavy lifting.

ZENODO

Immutable deposit of the code snapshot + its metadata record. Backed by CERN; long-term preservation policy.

SOFTWARE HERITAGE

Inria-led universal source-code archive. Archives software from many supported origins; verify ingestion for the specific repository. Gives every file a **SWHID** — content-addressed identifiers.

Software Heritage identifier shapes

SWHID

```
swh:1:dir:... # directory identifier shape
swh:1:rev:... # revision identifier shape
swh:1:snp:... # snapshot identifier shape

# Illustrative syntax, not issued identifiers.
# Check this repository's archival status.
```

archive.softwareheritage.orgzenodo.org

A citation that includes a **DOI + SWHID** is the gold standard: one for people, one for content-addressed preservation.

F + A · the minimum viable artefacts

F · FINDABLE

DOI (concept + version), rich machine-readable metadata, indexed by a registry.

A · ACCESSIBLE

HTTPS + git or a registry, auth only where strictly necessary, archive for persistence.

Your repo today has

`CITATION.cff`, `codemeta.json`, environment locks and preserved examples. The first DOI follows the reviewed release.

BRING TO PRACTICAL 1

FAIR metadata helps search engines describe and index software. Next: step into the shoes of a *consumer*. What does Betty actually find, and what evidence can you inspect directly?

02

Practical 1 · Discover research software

Experience findability from the **consumer** side —
search, sort, export, inspect.

F

A

I

R

Find the hunter–prey model in 12 minutes

- 1 Open the [Betty Research Engine](#).
- 2 Search for `fair-hunter-prey-simulation`, then `system dynamics`.
- 3 Inspect the search sources. Use GitHub search or sign-in where the service offers them.
- 4 Sort by citations if available. Record the result source and missing fields.
- 5 Export the result set as JSON if available; record the query and date.
- 6 Find this repository, or record the indexing gap and follow the [direct GitHub link](#).
- 7 Inspect authors, version, license and citation metadata. Follow the literature, both implementations, and Case 1/2 inputs.
- 8 Follow the [Zenodo publication evidence](#). Does a citation count measure scientific quality or only observed citations?

What made the hunter–prey model findable?

Prompt: Which fields did Betty expose? Which came from GitHub or Zenodo? Which were absent? Compare what the repository declares with what the search service actually indexed.

TITLE & AUTHORS

From `CITATION.cff` — ORCIDs make authorship machine-resolvable.

LICENSE & VERSION

SPDX identifiers and release versions describe reuse conditions and the source snapshot.

CITATION COUNT

A citation count depends on the index and the citing records it has collected.

TAKEAWAY

Rich metadata improves discoverability and interpretation. Missing search results are indexing evidence, not a verdict on the model. Follow direct references when a new repository is not yet indexed.

03

Interoperable research software

Software that **talks** to other software — open formats, qualified references, shared vocabularies.

F A **I** R

I1 · read, write, exchange via community standards I1

FAIR4RS · I1

Software reads, writes and exchanges data in a way that meets domain-relevant community standards.

PLAIN ENGLISH · If two independent pieces of software want to talk, they need shared formats (HDF5, NetCDF, CSV+schema) and shared vocabularies. Where APIs are involved, they should be documented with open specs (OpenAPI, AsyncAPI).

IN FAIR-HUNTER-PREY

- ✓ **Model sources:** text-based Vensim `.mdl` ; XML/XMILE in Stella `.stm` .
- ✓ **Inputs:** existing `.cin`, CSV and workbook files; shared scientific quantities, different representations.
- ✓ **Results:** CSV readers preserve headers, units and independent time axes before pandas/Matplotlib analysis.
- ✓ **Execution:** PySD translates copies of either implementation using the existing compatibility adapters.

An OpenAPI service remains an illustrative option, shown next.

Open formats > bespoke files I1

SCENARIO	BESPOKE	INTEROP-FRIENDLY	WHY IT MATTERS
Simulation output	<code>results.bin</code> (custom)	<code>results.h5</code> (HDF5) or <code>.nc</code> (NetCDF)	Every language has a reader; chunked & self-describing.
Tabular data	Excel with merged cells	CSV + a data dictionary	Parseable by any tool; diff-friendly under version control.
Service over HTTP	Undocumented JSON endpoint	OpenAPI spec + JSON-Schema'd responses	Auto-generate clients, stubs, tests; discoverable.
Configuration	<code>my_input.txt</code> (position-sensitive)	TOML / YAML with a JSON Schema	Validated, editable, idempotent.

I1 heuristic: *"If my format has a specification I can link to, and multiple implementations read it, it's interop-grade."*

Here: Vensim trajectories and historical data can have different time axes. Stella Case 2 contains final values only. A CSV extension alone does not define these semantics.

Example · OpenAPI beats a README I1

openapi.yaml · excerpt

3.1.0

```
openapi: 3.1.0
info: {title: Kaibab results API, version: 0.1.0}
paths:
  /results/{implementation}/{case}:
    get:
      summary: Inspect saved simulation evidence
      parameters:
        - {name: implementation, in: path, required: true,
          schema: {type: string, enum: [vensim, stella]}}
        - {name: case, in: path, required: true,
          schema: {type: string, enum: [case1, case2]}}
      responses:
        "200":
          description: Values, units, time axes and export kind
```

What you get for free

- Auto-generated **Python, R, Julia, TypeScript** clients.
- Swagger UI & Redoc **documentation** out of the box.
- Fuzz testing from the schema (`schemathesis`).
- Clients validate requests *before* they hit your service.

An OpenAPI spec is itself a machine-readable, qualified reference — it's **I1 + I2** in a single file.

Illustrative design; no HTTP service is implemented. A response must distinguish `trajectory` and `final-values`. The working interfaces are the Python API, CLI and CSV readers.

I2 · qualified references to other objects I2

FAIR4RS · I2

Software includes qualified references to other objects.

PLAIN ENGLISH · Every *external thing* the software mentions — an author, a licence, a dataset, an instrument, a vocabulary term — should be referenced through a resolvable identifier from a maintained authority, not a free-text string.

I2 vs R2. I2 covers references to *anything other than software*. R2 (later) is the twin principle for references to *other software*.

IN FAIR-HUNTER-PREY

- ✓ Authors: verified [Raphael Ginster ORCID](#) and [Vasiliy Seibert ORCID](#); no invented identifier for Matthias Papesch.
- ✓ Original model: [Deaton & MacDonald, System Dynamics Learning Guide](#).
- ✓ Licensing: SPDX identifiers and [per-file mappings](#).
- ✓ Scientific meaning: [predator-removal concept](#) linked to declarations and assignments.

"Alice" vs "ORCID 0000-0001-..." I2

Unqualified

ambiguous

```
authors: [Alice, Bob]
license: "MIT"
depends_on: [numpy]
```

- Which Alice?
- MIT — the variant? MIT-0? MIT with advertising clause?
- Which *version* of NumPy, how constrained?

Qualified

unambiguous

```
authors:
  - name: Alice Beech
    orcid: https://orcid.org/0000-0001-2345-6789
license:
  sdx: MIT
  url: https://spdx.org/licenses/MIT.html
dependencies:
  - name: numpy
    pypi: https://pypi.org/project/numpy/
    version: ">=1.26,<2.0"
```

Every claim now points to an external authority — ORCID, SPDX, PyPI. Machines can cross-check.

Author/license references illustrate I2. The dependency row illustrates R2, the specific principle for other software.

codemeta.json — line by line I2

codemeta.json

JSON-LD

```
{
  "@context": "https://doi.org/10.5063/schema/codemeta-2.0",
  "@type": "SoftwareSourceCode",
  "name": "fair-hunter-prey-simulation",
  "version": "0.7.0",
  "codeRepository": "https://github.com/FAIR-systemDynamics/fair-hunter-prey-simulation",
  "programmingLanguage": "Python",
  "runtimePlatform": [
    "Python 3.12",
    "Python 3.13"
  ],
  "author": [
    {
      "@type": "Person",
      "name": "Raphael Ginster",
      "id": "https://orcid.org/0000-0003-2394-7064"
    },
    {
      "@type": "Person",
      "name": "Matthias Papesch"
    },
    {
      "@type": "Person",
      "name": "Vasiliy Seibert",
      "id": "https://orcid.org/0000-0002-7121-6816"
    }
  ]
}
```

Every URL identifies its subject

- ✓ Repository URL — current source and collaboration.
- ✓ Version and DOI — the released research artifact.
- ✓ SPDX — per-file license semantics.
- ✓ ORCID — verified person identity.
- ✓ Literature and concept URI — scientific context.
- ✓ PyPI references — dependencies, under R2.

[Complete CodeMeta file](#) · [Zenodo creators and related identifiers](#)

← Lecture 3 · CodeMeta & Metadata4Ing

Controlled vocabularies eliminate ambiguity · and give credit I2

The ambiguity problem

Two files name **“Fraction Predators Killed per Year”**. Does a value describe the scientific quantity, a model declaration, or a scenario assignment?

- Scientific concept: annual predator-removal fraction, with its declared meaning and unit.
- Case 1 assignment: **0**; Case 2 assignment: **0.2**.
- Vensim and Stella encode those assignments in different files.

The credit problem

A resolvable concept URI identifies the definition being reused. Cite the literature and vocabulary source rather than leaving the meaning implicit.

← Lecture 3 · controlled vocabularies for data

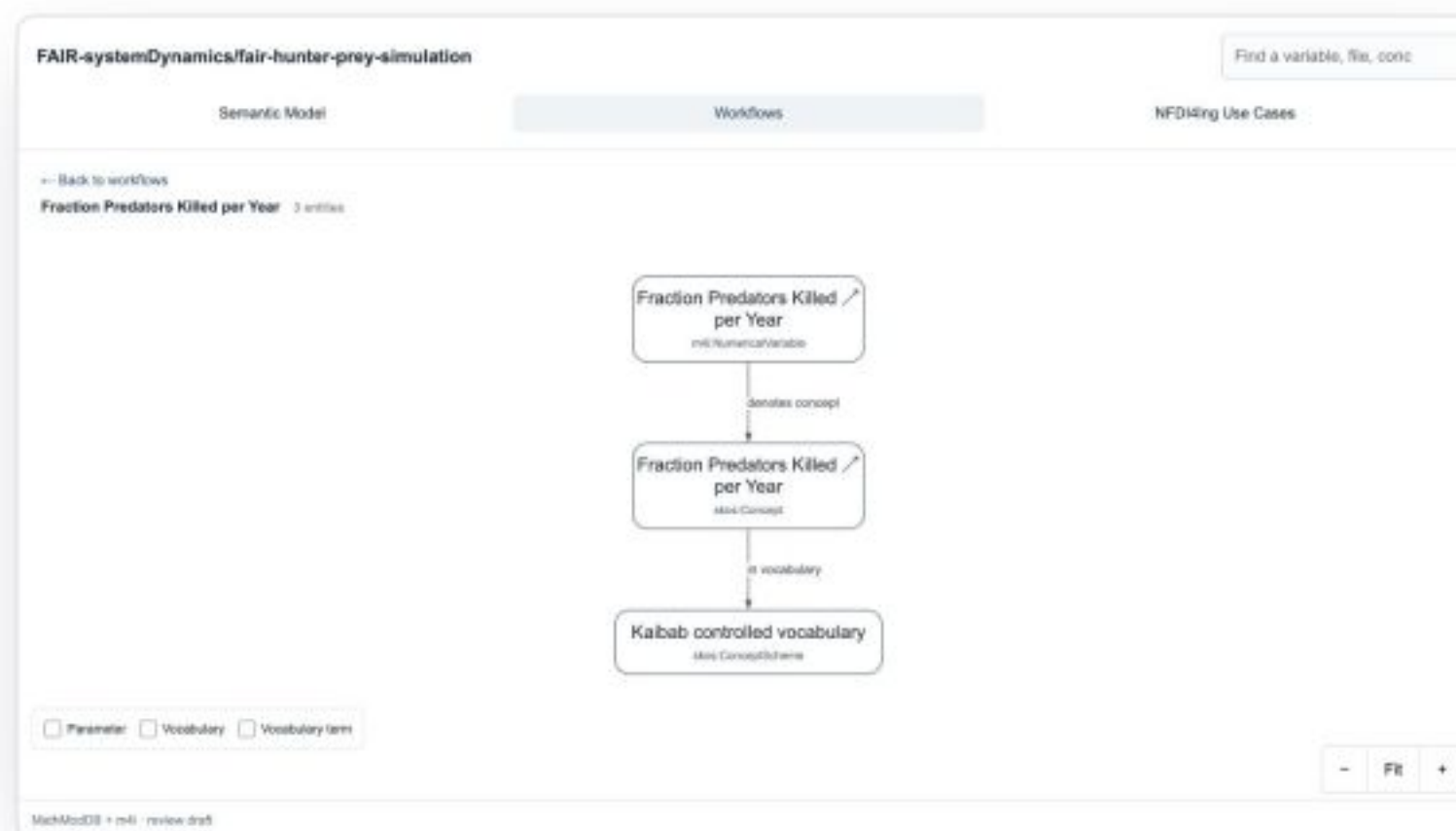
Case 2 · two bindings of one concept

text

```
Vensim .cin: Fraction Predators Killed per Year = 0.2
Stella CSV: Fraction Predators Killed per Year,0.2
```

The selected assignment is hashed into each run record; the inspected data keeps its implementation and case identity.

Semantic model · controlled vocabulary



Local vocabulary term, explicitly a review draft. [Inspect full-size screenshot](#)

[Open the concept](#) · [Follow its configuration binding](#)

Case 2: [Vensim .cin](#) · [Stella CSV](#)

Dependencies are *also* qualified references R2

Unqualified dependency

✗ ambiguous

```
dependencies = ["pandas"]
```

"I depend on pandas." But which version? Today's? Next year's? If pandas 3.0 breaks my code and someone installs it, whose fault is it?

Qualified dependency

✓ specific

```
dependencies = ["pandas>=2.2,<3"]
```

"I depend on pandas, specifically in this API-stable range." That declares the supported range. A machine can check it. A future reuser knows the constraint.

This is R2 because

1. **It's a reference** → Points to something external (pandas package on PyPI)
2. **It's qualified** → With precise version constraints
3. **It's from a maintained authority** → PyPI hosts pandas; the version numbers are real and resolvable
4. **It's machine-readable** → Any packaging tool (pip, uv, poetry, hatch) understands `>=2.2,<3`

PEP 621 is a public standard, so every tool reads it the same way. No ambiguity.

pyproject.toml · from source to installed package R2

pyproject.toml · core sections

TOML

```
[build-system]
requires = ["setuptools>=77,<83", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "fair-hunter-prey"
version = "0.7.0"
requires-python = ">=3.12,<3.14"
dependencies = [
    "pysd==3.14.3", "pandas>=2.2,<3",
    "matplotlib>=3.8,<4", "openpyxl>=3.1,<4",
]
```

Five steps to pip install

1. **Create pyproject.toml** — Define build-system, name, version, dependencies, authors, license.
2. **Organize code** — Use `src/fair\_hunter\_prey/` layout; distribution: fair-hunter-prey; import: fair_hunter_prey.
3. **Add `\_\_init\_\_.py`** — Makes the directory a Python package; export main classes/functions.
4. **Install locally** — `pip install -e .` tests installability before publishing; `-e` = editable mode.
5. **Publish to PyPI** (optional) — `python -m build` then `twine upload dist/\*`; now anyone runs `pip install fair-hunter-prey`.

Alternative: Direct from GitHub

No PyPI account needed. Pin to the reviewed commit; use v0.7.0 after publication:

```
python -m pip install "fair-hunter-prey[teaching] @ git+https://github.com/FAIR-systemDynamics/fair-hunter-prey-simulation@ca6c7c85df3335e7197b1ef1bfd75af2ea41a33a"
```

Trade-offs: GitHub is immediate & automatic (an installable commit can be pinned); PyPI is discoverable & cached globally. Both are R2: qualified references to a maintained repository.

[Complete pyproject.toml](#) · Both native implementations are bundled without reorganizing the scientific source tree.

04

Practical 2 · Test interoperability

Can you actually **run** a stranger's software — and does it break when dependencies change?

F

A

I

R

Install, run, break, understand

- 1 Open [NFDI4Ing Jupyter](#) with Python 3.12/3.13. Install the [pinned teaching package](#).
- 2 Open [02_interoperability.ipynb](#). First inspect saved CSV exports with the unchanged readers, pandas and Matplotlib. This performs no simulation.
- 3 Run both implementations and both cases through **PySD**. Compare each with its own reference, preserving numerical methods and tolerances.
- 4 Inspect package versions. Explain what an absent dependency constraint or a misread final-values export could break. Do not modify the original research inputs.

Notebook · inspect exports, then execute with PySD

Python 3.12 / 3.13

```
from fair_hunter_pre import inspect_results, run_example

inspect_results("out/inspection") # saved exports only
for implementation in ("vensim", "stella"):
    for case in ("case1", "case2"):
        run_example(implementation, case,
                    f"out/{implementation}-{case}")
```

Why R2 (qualified dependencies) matters

Prompt: You ran code written by someone else, in an environment you don't control. What made that possible? What would have broken it?

PINNED VERSIONS

The repo specifies `pandas>=2.2,<3`. That's R2: a qualified, maintained reference to another software package.

STANDARD PACKAGE FORMAT

`pyproject.toml` is PEP 621. Every tool reads it the same way. No ambiguity.

OPEN PROTOCOL

`pip install` via GitHub uses standard tools on supported platforms. The Python route is independent; native authoring still uses vendor tools.

TAKEAWAY

■ Interoperability is not magic. It's: standards + constraints + open protocols.

Inspecting a CSV and executing a model are different activities. Stella Case 2's native export contains final values only; a new PySD trajectory is new execution evidence.

05

Reusable research software

Understandable, modifiable, **sustainable**. The biggest block in spirit — we triage to the highest-leverage artefacts.

F A I **R**

R1 · R1.1 · R1.2 · describe, license, trace

FAIR4RS · R1 · R1.1 · R1.2

R1. Software is described with a plurality of accurate and relevant attributes.

R1.1. Software is given a clear and accessible license.

R1.2. Software is associated with detailed provenance.

PLAIN ENGLISH · A reuser needs enough labels to evaluate the software *before* cloning it — what it does, how it's licensed (R1.1 · machine-readable SPDX), and the story of who built it and why (R1.2 · who, when, what changed).

IN FAIR-HUNTER-PREY

- ✓ [README.md](#), [docs/](#) and [examples/](#) — layered documentation, R1.
- ✓ [REUSE.toml](#) — MIT code, CC BY 4.0 documentation, CC BY-NC-SA 4.0 model-derived artifacts, R1.1.
- ✓ [CHANGELOG.md](#) and preserved teaching tags — the human-readable history, R1.2.
- ✓ Execution provenance records the implementation, case, PySD engine, inputs, environment and output hashes.
- ✓ The [FAIR4RS evidence table](#) links each claim to its evidence.

Documentation is layered R1

README.MD

Start here. Install, quick-start, what this project is *not*. < 2 screens.

DOCS/

How it works. Mathematical model, architecture, data formats. Static site: MkDocs, Sphinx, Quarto.

EXAMPLES/

How to use it. Runnable notebooks or scripts reproducing headline figures.

CHANGELOG.MD

How it changed. *Keep-a-Changelog* + semver give reusers an upgrade path.

CONTRIBUTING.MD

How to help. Even "accept issues only" is a useful signal.

FAIR4RS.MD (OPTIONAL)

An explicit self-audit — the linked FAIR4RS evidence table records this repository's status.

[Open the FAIR4RS evidence table](#)

No license ≠ public domain R1.1

The SPDX License List

- MIT, Apache-2.0, BSD-3-Clause — permissive.
- GPL-3.0-or-later, AGPL-3.0-only — copyleft.
- CC-BY-4.0 — for docs, *not* for code.

SPDX **identifiers are strings** — *that* is what lets a tool reason about compatibility automatically.

• spdx.org/licenses

Here: [REUSE maps each file](#) to MIT, CC BY 4.0, or the inherited CC BY-NC-SA 4.0 model license. Vensim and Stella retain their own proprietary terms.

YOU WANT...	GOOD DEFAULT
Max adoption	MIT or Apache-2.0
Improvements returned	GPL-3.0-or-later
Server-side reciprocity	AGPL-3.0-only
Scientific paper figures	CC-BY-4.0 (data/prose, not code)

No LICENSE file = all rights reserved. Public visibility alone grants no general reuse license; applicable permissions and exceptions still matter.

Who · what · when · why — for free R1.2

`git log · fair-hunter-prey`

provenance

```
207ca01 fix: identify PySD explicitly in execution output and logs
2648c8c docs: record FAIR verification and presentation review evidence
fd82aa2 ci: install the teaching package before the full existing test suite
eaa529e feat: add FAIR4RS lecture and reproducible hunter-prey workflows

# Earlier teaching milestones remain intact.
```

Descriptive commit messages support a changelog draft; the changelog records the curated story.

CHANGELOG.md

Keep-a-Changelog

```
# Abridged; read the complete changelog below
## [0.7.0] — release candidate
### Added
- Both implementations and both cases via PySD.
- Saved-export inspection and R0-Crate examples.
- FAIR4RS lecture and evidence map.
### Changed
- Restore all 52 reference lessons and styling.
### Preserved
- Models, configurations, readers and tolerances.
```

- `git log` — the raw trail.
- `CHANGELOG.md` — the curated story.
- `AUTHORS` / `.zenodo.json` — attribution persistence.

[Read the full versioned changelog](#) · [Inspect actual commits](#)

R2 · qualified references to other software R2

FAIR4RS · R2

Software includes qualified references to other software.

PLAIN ENGLISH · Every dependency your code needs to compile or run must be referenced through an authoritative source — name + version range + registry — not a loose "depends on NumPy". The reference itself should resolve to the qualifying authority.

R2 vs I2. I2 is about refs to *anything other than software*; R2 is specifically the software case.

IN FAIR-HUNTER-PREY

- ✓ `pysd==3.14.3` — translation and simulation with documented feature limits.
- ✓ `pandas>=2.2,<3`, `matplotlib>=3.8,<4`, `openpyxl>=3.1,<4`.
- ✓ `requires-python = ">=3.12,<3.14"`; exact tested teaching locks for both interpreters.
- ✓ Authority: [PyPI](#) and [PySD documentation](#).
- ✓ Native Vensim and Stella are separate software dependencies for native authoring/execution.

Package outputs identify PySD and the existing compatibility adapters.

Pin the whole world, not just the code R2

WEAKEST · DECLARE

`pyproject.toml`, `requirements.txt` with version ranges. Declares the supported dependency contract.

STRONGER · PIN

`poetry.lock`, `uv.lock`, `conda env export` — every transitive dep at a concrete version. Here: `teaching-py312.txt` and `teaching-py313.txt`.

STRONGEST · CONTAIN

`Dockerfile` / `apptainer.def` + image pushed to GHCR. Records additional OS/runtime layers when actually built and identified. No container image is published here.

Dockerfile · reuse-grade

OCI

```
# Illustrative Dockerfile; not a published image
FROM python:3.13-slim
# For a frozen build, record a verified base-image digest.
COPY dist/fair_hunter_pre-0.7.0-py3-none-any.whl /tmp/
RUN pip install /tmp/*.whl
ENTRYPOINT ["fair-hunter-pre"]
```

R3 · meet domain-relevant community standards R3

FAIR4RS · R3

Software meets domain-relevant community standards.

PLAIN ENGLISH · Use the packaging, testing, documentation, and distribution conventions of your language / community. Skipping them — even with otherwise beautiful code — makes reuse needlessly hard and signals "experimental" to reviewers.

IN FAIR-HUNTER-PREY

- ✓ **Packaging:** PEP 621 pyproject.toml and an installed-package check.
- ✓ **Citation:** CFF and CodeMeta with verified creators and references.
- ✓ **Tests:** existing scientific tests plus four implementation/case combinations.
- ✓ **CI:** Python 3.12 and 3.13; unchanged reference tolerances.
- ✓ **Licensing:** REUSE and SPDX, preserving the inherited model terms.

Let CI answer "does it still run?" R3

Distribute where your community looks

- PyPI / conda-forge — Python.
- CRAN — R.
- CPAN, Julia General, Rust crates.io, ...
- Otherwise: a **tagged GitHub release** + a DOI remains installable for years.

[Read the complete workflow](#) · CI also checks protected-file hashes and both Pages surfaces.

.github/workflows/ci.yml

smoke test

```
name: FAIR teaching verification
on: [push, pull_request]
jobs:
  verify:
    strategy:
      matrix: {python: ["3.12", "3.13"]}
    runs-on: ubuntu-latest
    steps:
      - checkout and locked environment setup
      - run: python -m pytest -q
      - run: python tools/fair/check_stella.py
      - run: python tools/fair/check_materials.py
      - run: python tools/fair/execute_notebooks.py
      - build and test wheel outside the checkout
```

A green badge on the README > two paragraphs of installation docs.

Write the sustainability plan *before* the code rots

Reusability two years in is not an artefact — it's a *policy*. Who keeps the tests green? Which Python version stays supported? What triggers a new Zenodo deposit?

The community answer is the **Software Management Plan (SMP)** — the software cousin of the DMP we wrote in Lecture 2.

← LECTURE 2 · RDMO SMP CATALOGUE

RDMO SMP — WHAT IT CAPTURES

Roles · maintainer, reviewer, release manager.

Versioning · semver + cadence.

Archival · Zenodo + Software Heritage.

Support · issue SLA, EoL policy.

Deprecation · how & when code is retired.

• rdmo.nfdi4ing.de

This repository's [release checklist](#) records the process. Maintainer responsibilities and support promises must be agreed with the collaborators.

R · the smallest thing that could break you

BRING TO PRACTICAL 3

You have inspected saved exports and run an implementation in a fresh Python environment. Now package its source, selected case, actual outputs, licenses and environment together.

In a disposable copy of the crate, change an output or remove an input. Re-run the hash check and explain why it fails. Discuss how missing licenses and unconstrained dependencies would obstruct reuse.

RO-Crate · research object in one container R

What is an RO-Crate?

A **standardized, portable directory structure** that bundles:

- **Code** — your source files
- **Data** — inputs, examples, outputs
- **Metadata** — authors (ORCID), license (SPDX), description
- **Environment** — dependencies, Python version, system reqs
- **Provenance** — git history, changelog, execution trace

All wrapped in a machine-readable manifest: `ro-crate-metadata.json` (JSON-LD).

Why you need it

REPRODUCIBILITY

The crate collects the available context needed to attempt regeneration years later. Not scattered across GitHub, Zenodo, supplementary materials, and lost emails.

PORTABILITY

Inspect the same standard crate across tools. Execution still requires compatible software and dependencies. This practical uses Jupyter.

PRESERVATION

Deposit a research-object archive in a suitable repository such as Zenodo; verify source-code archiving separately. Future tools understand RO-Crate structure — your work isn't locked into one archive format.

06

Practical 3 · Capture reusability via RO-Crate

Package your entire research object — code, data, metadata, environment, provenance — for reproducibility across time and tools.



Package hunter–prey as a Research Object

- 1 In [Jupyter](#), open [03_ro_crate.ipynb](#) using the same locked teaching environment.
- 2 Select one implementation and case. Generate a fresh PySD run into a separate output directory.
- 3 Create the crate. It includes the model, inputs, references, licenses, recorded environment, actual outputs and execution provenance.
- 4 Inspect `ro-crate-metadata.json`. Follow creator identifiers and find the selected scenario.
- 5 Verify file hashes; alter only a disposable copy to demonstrate detection of changed evidence.

Jupyter · actual execution and packaging

python

```
from fair_hunter_prex import run_example, create_crate
run_example("stella", "case2", "out/crate-run")
crate = create_crate("out/crate-run", "out/research-object")
print(crate / "ro-crate-metadata.json")
```

The adapter uses [rocrate](#) to describe files, authors and provenance.

The R in FAIR4RS · Reusability across time

Prompt: An RO-Crate captures your code, data, metadata, and environment together. Why does this matter for the future? What could break if you *didn't* package it this way?

REPRODUCIBILITY

This crate records the observed execution context: code version, deps, data, env. Identify what was actually run and assess future execution requirements.

PROVENANCE

Metadata (author ORCID, license, citations) travels with the code. Credit is preserved.

PORTABILITY

Standard format, many tools understand it. Not locked into one platform or archive.

TAKEAWAY

- R (Reusable) means: preserving the context needed for future reuse. RO-Crate is the container.

The FAIR4RS checklist — one page

F

FINDABLE

- Concept + version DOI on Zenodo
- CITATION.cff at root
- codemeta.json JSON-LD
- Check actual indexing in a registry or discovery service

A

ACCESSIBLE

- Installable: `pip install` via GitHub or PyPI
- `pyproject.toml` with PEP 621 metadata
- HTTPS + git (open protocols, no login walls)
- Verify Zenodo deposit and Software Heritage ingestion

I

INTEROPERABLE

- Open formats (HDF5, CSV, OpenAPI)
- Qualified references: ORCID, SPDX, ROR, vocabulary URIs
- Documented CSV meanings, units and time axes
- Standard package structure + standard metadata

R

REUSABLE

- LICENSE (SPDX id) + README + examples
- CHangelog.md + conventional commits + semver tags
- CI green badge + RDMO Software Management Plan
- RO-Crate packaging for reproducibility across time
- Qualified software references and tested dependency locks (R2)

Use the [FAIR4RS evidence table](#) to distinguish implemented, verified and pending items. This checklist supports assessment; it is not a certification.

The services you touched today

BETTY RESEARCH ENGINE

Find research artefacts by federated search across NFDI.

- `software.nfdi4ing.de`

TERMINOLOGY SERVICE

Describe with resolvable URIs for engineering concepts.

- `terminology.nfdi4ing.de`

JUPYTER SERVICE

Run & reuse any pip-installable package in a sharable notebook.

- `jupyter.nfdi4ing.de`

RDMO

Write your (Software) Management Plan before the code rots.

- `rdmo.nfdi4ing.de`

COSCINE

Project data storage with PIDs and access control (Lectures 1–2).

- `coscine.de`

ING.GRID

OA journal for engineering RDM — a landing spot for this kind of work.

- `ing.grid`

Across the series: data + software, together, is reproducible research.



Your repo, next week — FAIR4RS-shaped.

Start with the one thing you know is missing. Ship the rest incrementally.

Authors: Raphael Ginster · Matthias Papesch · Vasiliy Seibert

POINTERS

[FAIR4RS Principles](#)
[Software Heritage](#)
[SPDX License List](#)

STANDARDS

[CITATION.cff](#)
[CodeMeta crosswalk](#)
[ORCID](#) · [ROR](#)

THIS DECK AND EXAMPLE

[Hunter-prey slides \(this preview\)](#)
[GitHub repository](#)
[Concept DOI: awaiting first release](#)
[Original awesome-sim lecture](#)
CC BY 4.0 lecture; inherited model licensing

Repository evidence

The screenshot shows the GitHub repository page for 'fair-hunter-prey-simulation'. At the top, there are tabs for 'README', 'Contributing', and 'License'. Below the repository name, there are several status badges: 'FAIR teaching verification passing', 'Zenodo awaiting first release', 'Python 3.12 | 3.13', and 'licensing REUSE'. The authors listed are Raphael Ginster, Matthias Papesch, and Vasiliy Seibert. The description states it is a shared Kaibab scientific model implemented in Vensim and Stella Architect, with Case 1 and Case 2. It mentions Open CSV inspection and independent PySD execution. A link to the original model is provided. Below this, there are links to the semantic model and vocabulary, FAIR4RS teaching material, a review PR and pages preview, and a 52-slide adaptation register. A paragraph explains that the semantic site is live and unchanged, while revised slides are under review. A 'Quick start' section provides instructions on using Python 3.12 or 3.13 and installing the 'moving review branch'. A code block shows the commands to install the package and run the simulation. The bottom of the screenshot shows the start of a paragraph about inspection reads.

README Contributing License

fair-hunter-prey-simulation

FAIR teaching verification **passing** Zenodo awaiting first release Python 3.12 | 3.13 licensing **REUSE**

Authors: [Raphael Ginster](#), [Matthias Papesch](#), [Vasiliy Seibert](#).

A shared Kaibab scientific model, implemented in **Vensim** and **Stella Architect**, with **Case 1** and **Case 2**. Open CSV inspection and independent **PySD** execution make the artifacts inspectable and reusable alongside the proprietary native workflows. Original model: [Deaton and MacDonald, System Dynamics Learning Guide \(2025\)](#).

[Semantic model and vocabulary](#) · [FAIR4RS teaching material](#) · [Review PR and Pages preview](#) · [52-slide adaptation register](#)

The semantic site is live and unchanged. The revised slides are under review; `/slides/` will be published only after a future merge. The first Zenodo release requires review of its exact commit; no DOI is claimed before publication.

Quick start

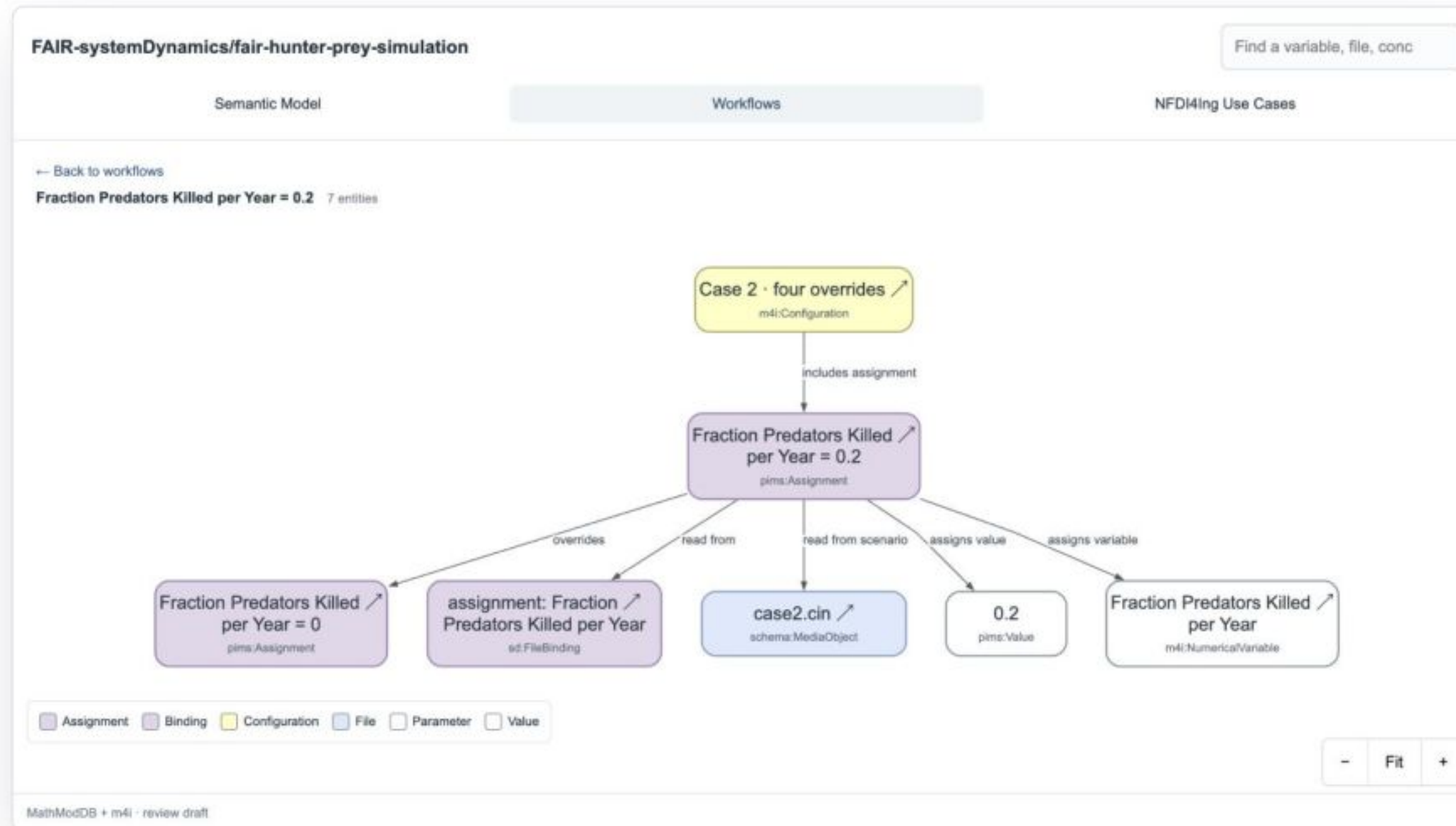
Use Python 3.12 or 3.13. This command installs the **moving review branch**; for an exact rerun, use the full commit shown in the PR or the commit-pinned installation command in its slide preview.

```
python -m pip install "fair-hunter-prey[teaching] @ git+https://github.com/FAIR-systemDynamic
fair-hunter-prey inspect --output-dir out/inspection
fair-hunter-prey run --implementation stella --case case2 --output-dir out/stella-case2
fair-hunter-prey crate --run-dir out/stella-case2 --output-dir out/research-object
```

Inspection reads saved exports; `run` uses PySD and records the selected implementation and case. See the

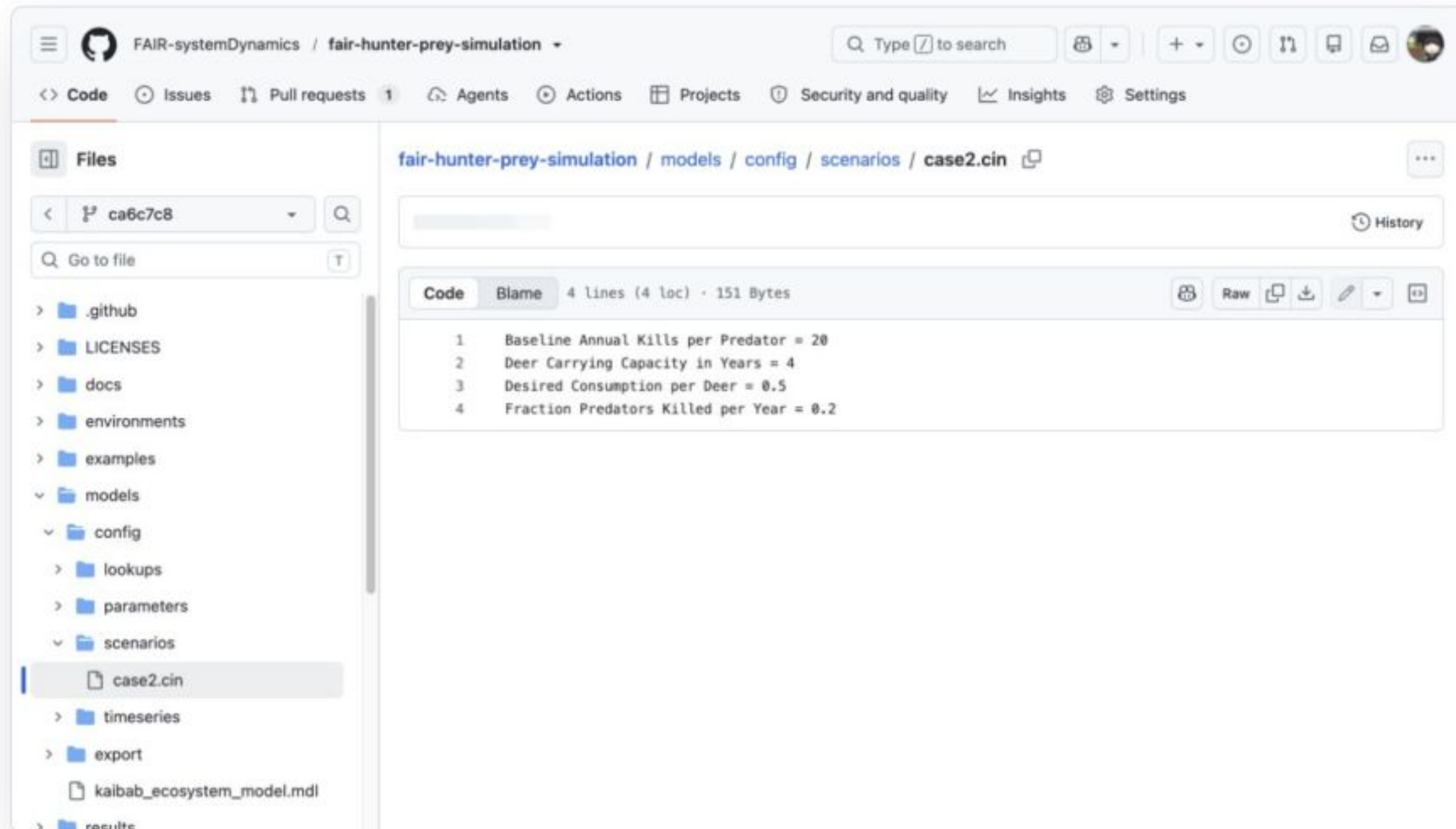
The reviewed README identifies authors, examples, licensing and publication status. [Open full-size image](#)

A value and its configuration binding



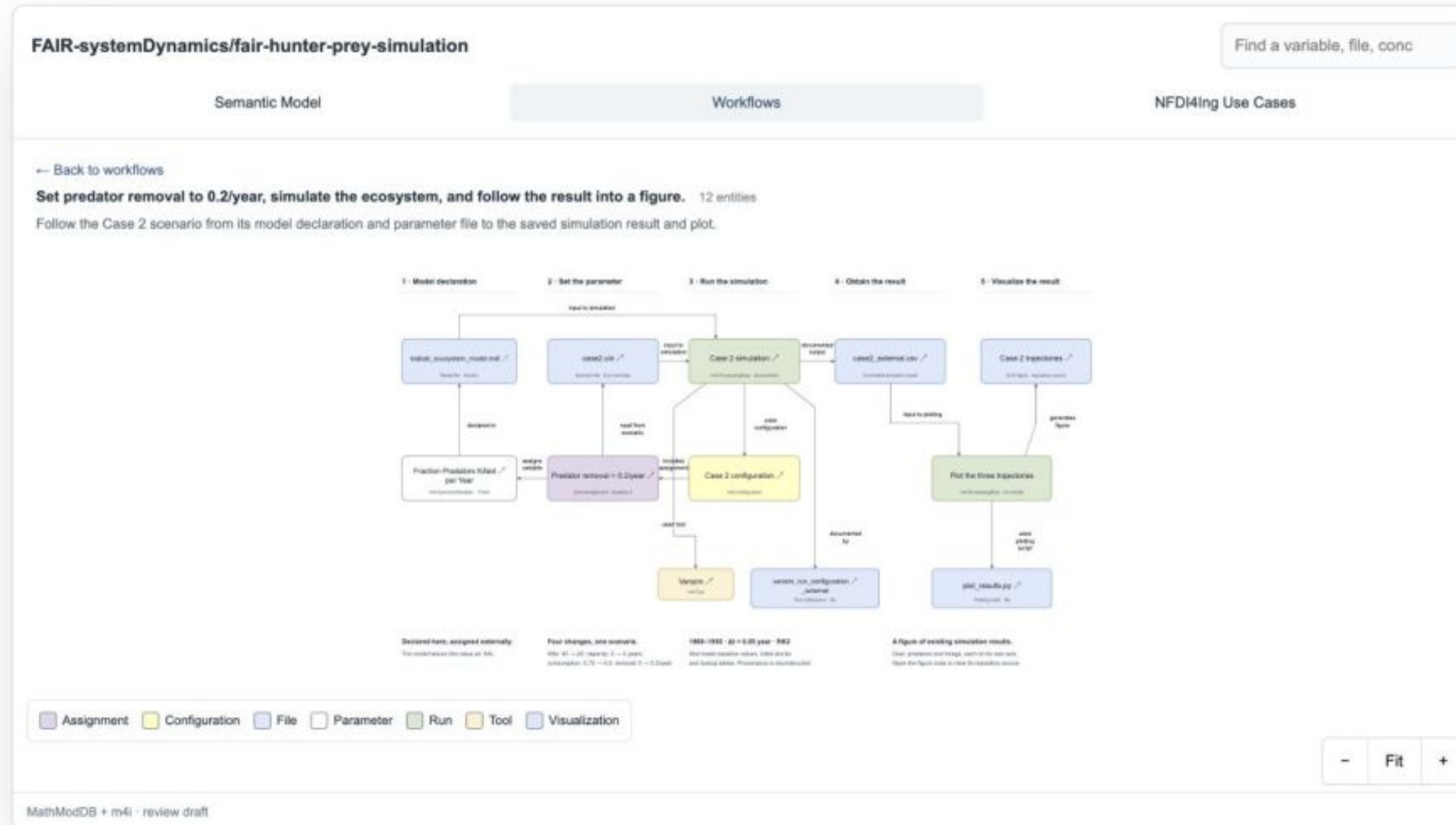
The assignment, parameter and source file are distinct entities. [Open full-size image](#)

The original Case 2 changes file



Four changes, preserved exactly as the collaborators recorded them. [Open full-size image](#)

From configuration to an inspected figure



The existing semantic workflow labels native provenance as reconstructed. [Open full-size image](#)

Executed interoperability notebook

3. Execute each implementation and case independently

PySD translates model copies using the repository's unchanged compatibility adapters. Vensim uses RK2 midpoint; Stella uses RK2 Heun with held STEP evaluation. Both integrate with the model's 0.05-year time step. Vensim saves annual samples to match the established reference check. These are **new PySD runs**, not native application runs. PySD support has feature limits; see its [documentation](https://pysd.readthedocs.io/en/latest/).

```
runs = {}
records = []
for implementation in ("vensim", "stella"):
    for case in ("case1", "case2"):
        result = run_example(implementation, case, output / f"{implementation}-{case}")
        runs[implementation, case] = result
        record = json.loads(result["provenance"].read_text())
        records.append({key: record[key] for key in ("implementation", "case", "engine", "method", "save_interval_years",
"revision")})
display(pd.DataFrame(records))
```

	implementation	case	engine	method	save_interval_years	revision
0	vensim	case1	PySD	rk2-midpoint	1.00	ca6c7c85df3335e7197b1ef1bfd75af2ea41a33a
1	vensim	case2	PySD	rk2-midpoint	1.00	ca6c7c85df3335e7197b1ef1bfd75af2ea41a33a
2	stella	case1	PySD	rk2-heun	0.05	ca6c7c85df3335e7197b1ef1bfd75af2ea41a33a
3	stella	case2	PySD	rk2-heun	0.05	ca6c7c85df3335e7197b1ef1bfd75af2ea41a33a

4. Inspect execution evidence

Compare each run with its own native reference, as done by the repository tests. The original tolerance is 0.1% for Vensim Case 1 and 5% for Case 2 on the established annual comparison grid. Stella's reference comparison uses relative tolerance 1e-9; its Case 2 supports only a final-value comparison. Do not require identical trajectories between tools.

Executed on Python 3.13: both implementations, both cases, PySD methods and source revision. [Open full-size image](#)